



本科实验报告

课程名称:	计算机网络
姓 名:	刘仁钦
学 院:	计算机科学与技术学院
专 业:	计算机科学与技术
学 号:	3230106230
指导教师:	韩劲松

2025 年 12 月 08 日

目录

一、 实验目的	3
二、 实验内容	3
三、 主要仪器设备	3
四、 操作方法与实验步骤	3
1. 实现 TCPConnection	3
1.1. 成员变量与初始化	3
1.2. 核心方法实现	4
1.2.1. <code>segment_received</code>	4
1.2.2. <code>write</code>	4
1.2.3. <code>tick</code>	4
1.2.4. <code>_push_segments_out</code>	5
2. 整合应用层 Webget	5
五、 实验数据记录和处理	5
1. 实验结果	5
1.1. 问题一	5
1.2. 问题二	6
1.3. 问题三	7
1.4. 问题四	8
1.5. 问题五	9
2. 思考题	9
2.1. 问题一	9
2.2. 问题二	10
2.3. 问题三	10
2.4. 问题四	10
六、 讨论、心得	10
七、 附录	11
1. <code>libsponge/tcp_connection.hh</code>	11
2. <code>libsponge/tcp_connection.cc</code>	14
3. <code>apps/webget.cc</code>	18

浙江大学实验报告

实验项目名称: Lab5 综合实验: 完整的 TCP 实现

学生姓名: 刘仁钦 专业: 计算机科学与技术 学号: 3230106230

一、实验目的

- 学习掌握 TCP 的工作原理
- 学习掌握 TCP connection 的相关知识
- 学习掌握协议栈结构

二、实验内容

- 将 TCPSender 和 TCPReceiver 结合, 实现一个 TCP 终端, 同时收发数据。
- 实现 TCP connection 的状态管理, 如连接和断开连接等。
- 整合网络接口、IP 路由以及 TCP 并实现端到端的通信。

三、主要仪器设备

- 联网的 PC 机
- Linux 虚拟机

四、操作方法与实验步骤

本次实验的核心任务是实现 TCPConnection 类, 它将之前实验中实现的 TCPSender 和 TCPReceiver 结合起来, 管理整个 TCP 连接的生命周期, 处理数据的收发以及状态的转换。此外, 还需要对 webget 程序进行修改, 使其支持我们自己实现的完整 TCP 协议栈。

1. 实现 TCPConnection

TCPConnection 是 TCP 协议的最终形态, 它作为一个“粘合剂”和“管理者”, 协调 Sender 和 Receiver 的工作。

1.1. 成员变量与初始化

在 libspoon/tcp_connection.hh 中, TCPConnection 维护了 _sender 和

_receiver 对象。此外, _segments_out 用于存放待发送的 TCP 段;

_linger_after_streams_finish 是一个布尔值, 用于判断在双向流结束后是否需要保持连接一段时间 (以等待对端可能的重传), 初始化为 true; _is_active 标记连接是否处于活跃状态, 初始化为 true; _time_since_last_segment_received 记录距离上一次收到段过去的时间, 用于超时判断。

1.2. 核心方法实现

在 `libsponge/tcp_connection.cc` 中，我们需要实现以下几个关键方法。

1.2.1. `segment_received`

处理入站数据段的函数。首先检查收到的段是否包含 `RST` 标志。如果是，说明连接异常，调用 `_unclean_shutdown(false)` 立即断开连接，将输入输出流标记为错误，并不再发送任何数据。

接着，将段传递给 `_receiver.segment_received(seg)`，让 `Receiver` 处理 `SYN`、`Payload` 和 `FIN`。如果段包含 `ACK` 标志，提取 `ackno` 和 `window_size`，传递给 `_sender.ack_received()`，让 `Sender` 更新状态并清理已确认的段。

如果在出站流结束前（`Sender` 未 `EOF`），入站流已经结束（`Receiver` 收到 `FIN`），说明是对端先关闭，我们是被动关闭。此时将 `_linger_after_streams_finish` 设为 `false`，意味着我们不需要在最后等待 `2MSL`。

最后，如果收到的段占据了序列号空间，我们需要发送一个 `ACK` 进行确认。如果收到的段是 `Keep-alive` 包（`seqno` 为当前 `ackno` - 1，长度为 0），我们也必须回复一个 `ACK`。这通常通过调用 `_sender.fill_window()` 尝试填充数据实现，如果 `Sender` 没有数据可发，则调用 `_sender.send_empty_segment()` 强制生成一个纯 `ACK` 包。最后调用 `_push_segments_out()` 将生成的段真正推入输出队列。

1.2.2. `write`

此方法用于应用程序向下层 TCP 写数据。实现很简单：直接调用

`_sender.stream_in().write(data)` 将数据写入 `Sender` 的字节流，然后调用 `_sender.fill_window()` 尝试将这些数据打包成 `Segment` 发送出去。

1.2.3. `tick`

此方法由操作系统定期调用，用于处理时间相关的逻辑。首先更新

`_time_since_last_segment_received`，然后调用 `_sender.tick()`，让 `Sender` 处理重传逻辑。

同时，需要检查 `_sender.consecutive_retransmissions()`，如果超过最大重传次数，调用 `_unclean_shutdown(true)` 强行断开，并发送 `RST` 给对端。

关于 clean shutdown, 需要检查是否满足三个条件: 入站流已结束、出站流已结束、所有发送的数据都已被确认。如果满足, 且 `_linger_after_streams_finish` 为 `false` (被动关闭), 直接设 `_is_active = false`。如果为 `true` (主动关闭), 则需等待 `10 * _cfg.rt_timeout` 时间后, 才设 `_is_active = false`。

1.2.4. `_push_segments_out`

这是我自己封装的一个辅助函数, 用于将 `_sender` 生成的段加工后放入 `_segments_out`。它会遍历 `_sender.segments_out()` 中的每一个段, 设置 `seg.header().win` 为当前 `_receiver.window_size()`。如果有 ACK, 则设置 `ack` 标志和 `ackno`, 最后将处理好的段加入 `_segments_out` 队列。

2. 整合应用层 Webget

为了验证完整的协议栈, 我们需要修改 Lab 2 中的 `webget` 程序, 使其不再使用操作系统提供的 Socket, 而是使用我们自己实现的包含 TCP/IP 协议栈的 Socket。

在 `apps/webget.cc` 中, 进行了如下修改: 首先引入头文件 `tcp_sponge_socket.hh`; 其次将 `TCPSocket` 替换为 `FullStackSocket`。`FullStackSocket` 是一个封装了完整协议栈 (TCP + IP + Ethernet 等) 的 Socket 类; 最后在 `get_URL` 函数末尾, 添加 `sock.wait_until_closed()`, 确保连接完全关闭后再退出程序。

具体代码修改将在后续实验结果部分展示。

五、实验数据记录和处理

1. 实验结果

1.1. 问题一

实现 TCPConnection 的关键代码截图

在这里展示 `libsponge/tcp_connection.cc` 的核心逻辑实现。

```

void TCPConnection::segment_received(const TCPSegment &seg) {
    if (!_is_active) return;

    _time_since_last_segment_received = 0;

    // 1. 如果设置了RST标志, 将入站流和出站流都设置为错误状态, 并永久终止连接。
    if (seg.header().rst) {
        _unclean_shutdown(false);
        return;
    }

    // 2. 把这个段交给TCPReceiver, 这样它就可以在传入的段上检查它关心的字段: seqno、SYN、负载以及FIN。
    _receiver.segment_received(seg);

    // 3. 如果设置了ACK标志, 则告诉TCPSender它关心的传入段的字段: ackno和window_size。
    if (seg.header().ack) {
        _sender.ack_received(seg.header().ackno, seg.header().win);
    }

    // 被动关闭检查: 如果在出站流发送结束前, 入站流已经全部接收完毕, 则需要将_linger_after_streams_finish设置为false
    if (_receiver.stream_out().input_ended() && !_sender.stream_in().eof()) {
        _linger_after_streams_finish = false;
    }

    // 4. 如果传入的段包含一个有效的序列号, TCPConnection确保至少有一个段作为应答被发送, 以反应ackno和window_size的更新。
    // 5. 如果传入的段包含一个无效的序列号, 这种段被称为“keep-alive”...TCPConnection应该回复这些“keep-alive”。
    bool send_ack_needed = seg.length_in_sequence_space() > 0;

    // 判断 keep-alive (seqno = ackno - 1, length = 0)
    if (_receiver.ackno().has_value() &&
        seg.length_in_sequence_space() == 0 &&
        seg.header().seqno == _receiver.ackno().value() - 1) {
        send_ack_needed = true;
    }

    if (send_ack_needed) {
        _sender.fill_window();
        if (_sender.segments_out().empty()) {
            _sender.send_empty_segment();
        }
    }

    _push_segments_out();
}

```

1.2. 问题二

运行 make check 命令的运行结果截图

```
1 make check
```

```
bash
```

运行 Lab 5 的完整测试集, 结果如下:

```

      Start 152: t_icnS_128K_8K_l
147/164 Test #152: t_icnS_128K_8K_l ..... Passed    0.40 sec
      Start 153: t_icnS_128K_8K_L
148/164 Test #153: t_icnS_128K_8K_L ..... Passed    0.34 sec
      Start 154: t_icnS_128K_8K_LL
149/164 Test #154: t_icnS_128K_8K_LL ..... Passed    0.39 sec
      Start 155: t_icnR_128K_8K_l
150/164 Test #155: t_icnR_128K_8K_l ..... Passed    0.59 sec
      Start 156: t_icnR_128K_8K_L
151/164 Test #156: t_icnR_128K_8K_L ..... Passed    0.26 sec
      Start 157: t_icnR_128K_8K_LL
152/164 Test #157: t_icnR_128K_8K_LL ..... Passed    0.69 sec
      Start 158: t_icnD_128K_8K_l
153/164 Test #158: t_icnD_128K_8K_l ..... Passed    0.64 sec
      Start 159: t_icnD_128K_8K_L
154/164 Test #159: t_icnD_128K_8K_L ..... Passed    0.25 sec
      Start 160: t_icnD_128K_8K_LL
155/164 Test #160: t_icnD_128K_8K_LL ..... Passed    0.73 sec
      Start 161: t_isnS_128K_8K_l
156/164 Test #161: t_isnS_128K_8K_l ..... Passed    0.23 sec
      Start 162: t_isnS_128K_8K_L
157/164 Test #162: t_isnS_128K_8K_L ..... Passed    0.35 sec
      Start 163: t_isnS_128K_8K_LL
158/164 Test #163: t_isnS_128K_8K_LL ..... Passed    0.47 sec
      Start 164: t_isnR_128K_8K_l
159/164 Test #164: t_isnR_128K_8K_l ..... Passed    0.35 sec
      Start 165: t_isnR_128K_8K_L
160/164 Test #165: t_isnR_128K_8K_L ..... Passed    0.26 sec
      Start 166: t_isnR_128K_8K_LL
161/164 Test #166: t_isnR_128K_8K_LL ..... Passed    1.51 sec
      Start 167: t_isnD_128K_8K_l
162/164 Test #167: t_isnD_128K_8K_l ..... Passed    0.39 sec
      Start 168: t_isnD_128K_8K_L
163/164 Test #168: t_isnD_128K_8K_L ..... Passed    0.33 sec
      Start 169: t_isnD_128K_8K_LL
164/164 Test #169: t_isnD_128K_8K_LL ..... Passed    0.50 sec

100% tests passed, 0 tests failed out of 164

Total Test time (real) = 41.48 sec
[100%] Built target check

```

图 2: make check 测试结果

测试结果显示所有测试用例均顺利通过，证明 `TCPConnection` 在建立连接、数据传输、重传、关闭连接各个阶段的状态流转和数据处理都是正确的。

1.3. 问题三

重新编写 `webget.cc` 的代码截图

展示修改后的 `apps/webget.cc`，使用了 `FullStackSocket`。

```

zju-comnet-labs > apps > c++ webget.cc
You, 2 minutes ago | 2 authors (cyrus and one other)
1  #include "socket.hh"
2  #include "util.hh"
3  #include "tcp_sponge_socket.hh"
4
5  #include <cstdlib>
6  #include <iostream>
7
8  using namespace std;
9
10 void get_URL(const string &host, const string &path) {
11     // Your code here.
12
13     // You will need to connect to the "http" service on
14     // the computer whose name is in the "host" string,
15     // then request the URL path given in the "path" string.
16
17     // Then you'll need to print out everything the server sends back,
18     // (not just one call to read() -- everything) until you reach
19     // the "eof" (end of file).
20
21     cerr << "Function called: get_URL(" << host << ", " << path << ").\n";
22     FullStackSocket sock;
23     sock.connect(Address{host, "http"});
24
25     string request{"GET " + path + " HTTP/1.1\r\nHost: " + host + "\r\nConnection: close\r\n\r\n"};
26     sock.write(request);
27
28     while (!sock.eof()) {
29         cout << sock.read();
30     }
31
32     sock.wait_until_closed();
33 }
34
35 int main(int argc, char *argv[]) {
36     try {
37         if (argc <= 0) {
38             abort(); // For sticklers: don't try to access argv[0] if argc <= 0.
39         }
40
41         // The program takes two command-line arguments: the hostname and "path" part of the URL.
42         // Print the usage message unless there are these two arguments (plus the program name
43         // itself, so arg count = 3 in total).
44         if (argc != 3) {
45             cerr << "Usage: " << argv[0] << " HOST PATH\n";
46             cerr << "\tExample: " << argv[0] << " stanford.edu /class/cs144\n";
47             return EXIT_FAILURE;
48         }
49         cyrus, 2 months ago + init
50         // Get the command-line arguments.
51         const string host = argv[1];
52         const string path = argv[2];
53
54         // Call the student-written function.
55         get_URL(host, path);
56     } catch (const exception &e) {
57         cerr << e.what() << "\n";
58         return EXIT_FAILURE;
59     }
60
61     return EXIT_SUCCESS;
62 }
63

```

图 3: 修改后的 webget 代码

1.4. 问题四

重新测试 webget 的测试结果展示

```
1 make check_webget
```

```
bash
```

运行针对 webget 的测试:


```

.../zju-comnet-labs/build main • ? } make check_webget
[100%] Testing webget...
Test project /home/cyrus28214/courses/Net/zju-comnet-labs/build
Start 31: t_webget
1/1 Test #31: t_webget ..... Passed 1.27 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) = 1.27 sec
[100%] Built target check_webget

```

图 4: webget 测试结果

测试通过，说明我们自己实现的 TCP 协议栈能够成功支持应用层程序（如 HTTP 客户端）进行实际的网络通信。

1.5. 问题五

最终测试中服务器和客户端的运行（连接、数据传输和结束）截图

启动模拟服务器和客户端进行通信测试：

Server:

```
1 ./apps/lab4 server cs144.keithw.org 3000
```

bash

Client:

```
1 ./apps/lab4 client cs144.keithw.org 3001
```

bash

图 5: 最终服务器-客户端通信测试

图中展示了客户端和服务端成功建立连接，并互相发送数据的过程（包括我的学号和姓名）。

2. 思考题

2.1. 问题一

ACK 标志的目的是什么？ackno 是经常存在的吗？

ACK 标志的目的是表明 TCP header 中的 `ackno` 字段是有效的。它用于确认已经收到的数据。

`ackno` 是经常存在的。只要连接建立后，双方都在不断的确认收到的数据，所以发出的每个 Segment 只要不是纯 `SYN` 都会设置 `ACK` 标志并携带 `ackno`。

2.2. 问题二

请描述 `TCPConnection` 的代码中是如何整合 `TCPReceiver` 和 `TCPSender` 的。

`TCPConnection` 作为一个更高级别的管理者，持有 `TCPSender` 和 `TCPReceiver` 的实例。接收方向：当 `TCPConnection` 收到一个 `TCPSegment` 时，它会调用 `_receiver.segment_received()` 让 `Receiver` 处理数据重组和序列号更新；同时，它检查 Segment 中的 `ACK` 信息，调用 `_sender.ack_received()` 让 `Sender` 更新滑动窗口和移除已确认的数据。发送方向：`TCPConnection` 通过 `write()` 接口接收上层应用数据，写入 `_sender` 的字节流。在发送 Segment 时，它会从 `_receiver` 获取当前的 `ackno` 和 `window_size`，填入 `TCPSegment` 的头部，从而实现捎带确认。

2.3. 问题三

在最终测试中服务器和客户端能互连吗？如果不能，你分析是什么原因？

能互连。从实验结果截图可以看到，客户端和服务端能够顺利完成三次握手进入 `ESTABLISHED` 状态，并且能够正确地双向传输文本数据。这说明底层的 TCP 协议栈实现（重传、窗口管理、序列号管理）以及网络层的路由转发都是正常的。

2.4. 问题四

在关闭连接的时候是否两端都能正常关闭？如果不能，你分析是什么原因？

能正常关闭。当一端（例如客户端）输入 `Ctrl+D` (EOF) 后，`Sender` 发送 `FIN` 包。对端收到 `FIN` 后，进入 `CLOSE_WAIT`（在我们的 FSM 模型中体现为入站流结束），并发送 `ACK`。代码中实现了 `_linger_after_streams_finish` 逻辑：主动关闭的一方在发送完 `FIN` 且收到对端的 `FIN` 后，会进入 `TIME_WAIT` 状态，等待 10 倍的超时时间以确保最后一个 `ACK` 到达；而被动关闭的一方在发送完自己的 `FIN` 并收到 `ACK` 后，因为检测到是对方先 `FIN`，会直接退出，不需要 `linger`。实验现象表明，双方都能正确经过四次挥手过程，最终程序退出。

六、讨论、心得

本次实验要求我们将之前分散实现的 TCP `Sender`、`Receiver` 以及网络层的 `Router` 和 `Interface` 全部整合在一起。

遇到的困难与解决:

1. `RST` 的处理: 一开始对 `RST` 的处理比较模糊, 通过阅读文档和测试用例失败的反饋, 明确了 `RST` 是对应“非正常关闭”或“严重错误”的情况, 需要立即切断连接。
2. TCP 的四次挥手状态机非常复杂, 需要精确判断四个条件: 进站结束、出站结束、字节流全确认、以及 `linger` 的逻辑。我通过仔细阅读文档在 `tick` 函数中准确实现了这一逻辑。
3. 在最终测试时, 由于模块众多, 定位 bug 比较困难。我学会了利用 `make check` 的输出日志, 结合 `DEBUG` 打印信息, 观察序列号的变化和状态机的流转, 从而定位问题。

心得:

实现一个完整的 TCP 协议栈让我对传输层有了极深的理解。以前书本上抽象的“滑动窗口”、“三次握手”、“拥塞控制 (虽然本实验简化了)”现在都变成了具体的代码逻辑。看到自己写的协议栈能够像真实的 TCP 一样支持 `webget` 下载网页, 支持客户端服务器聊天, 非常有成就感。

七、附录

1. `libsponge/tcp_connection.hh`

```
libsponge/tcp_connection.hh  cpp
1  #ifndef SPONGE_LIBSPONGE_TCP_FACTORED_HH
2  #define SPONGE_LIBSPONGE_TCP_FACTORED_HH
3
4  #include "tcp_config.hh"
5  #include "tcp_receiver.hh"
6  #include "tcp_sender.hh"
7  #include "tcp_state.hh"
8
9  //! \brief A complete endpoint of a TCP connection
10 class TCPConnection {
11     private:
12         TCPConfig _cfg;
13         TCPReceiver _receiver{_cfg.recv_capacity};
14         TCPSender _sender{_cfg.send_capacity, _cfg.rt_timeout,
15                             _cfg.fixed_isn};
16
17     //! outbound queue of segments that the TCPConnection wants sent
```

```

17     std::queue<TCPSegment> _segments_out{};
18
19     //! Should the TCPConnection stay active (and keep ACKing)
20     //! for 10 * _cfg.rt_timeout milliseconds after both streams have
    ended,
21     //! in case the remote TCPConnection doesn't know we've received its
    whole stream?
22     bool _linger_after_streams_finish{true};
23     bool _is_active{true};
24
25     size_t _time_since_last_segment_received{0};
26     void _trans_segments();
27     void _unclean_shutdown(bool send_rst);
28     void _push_segments_out();
29
30     public:
31         //! \name "Input" interface for the writer
32         //!@{
33
34         //! \brief Initiate a connection by sending a SYN segment
35         void connect();
36
37         //! \brief Write data to the outbound byte stream, and send it over
    TCP if possible
38         //! \returns the number of bytes from `data` that were actually
    written.
39         size_t write(const std::string &data);
40
41         //! \returns the number of `bytes` that can be written right now.
42         size_t remaining_outbound_capacity() const;
43
44         //! \brief Shut down the outbound byte stream (still allows reading
    incoming data)
45         void end_input_stream();
46         //!@}
47
48         //! \name "Output" interface for the reader
49         //!@{
50
51         //! \brief The inbound byte stream received from the peer
52         ByteStream &inbound_stream() { return _receiver.stream_out(); }
53         //!@}
54

```

```

55      //!< \name Accessors used for testing
56
57      //!<{
58      //!< \brief number of bytes sent and not yet acknowledged, counting
59      SYN/FIN each as one byte
60      size_t bytes_in_flight() const;
61      //!< \brief number of bytes not yet reassembled
62      size_t unassembled_bytes() const;
63      //!< \brief Number of milliseconds since the last segment was
64      received
65      size_t time_since_last_segment_received() const;
66      //!<< \brief summarize the state of the sender, receiver, and the
67      connection
68      TCPState state() const { return {_sender, _receiver, active(),
69      _linger_after_streams_finish}; };
70      //!<}
71
72      //!< \name Methods for the owner or operating system to call
73      //!<{
74
75      //!< Called when a new segment has been received from the network
76      void segment_received(const TCPSegment &seg);
77
78      //!< Called periodically when time elapses
79      void tick(const size_t ms_since_last_tick);
80
81      //!< \brief TCPSegments that the TCPConnection has enqueued for
82      transmission.
83      //!< \note The owner or operating system will dequeue these and
84      //!< put each one into the payload of a lower-layer datagram (usually
85      Internet datagrams (IP),
86      //!< but could also be user datagrams (UDP) or any other kind).
87      std::queue<TCPSegment> &segments_out() { return _segments_out; }
88
89      //!< \brief Is the connection still alive in any way?
90      //!< \returns `true` if either stream is still running or if the
91      TCPConnection is lingering
92      //!< after both streams have finished (e.g. to ACK retransmissions
93      from the peer)
94      bool active() const;
95      //!<}
96
97      //!< Construct a new connection from a configuration

```

```

90     explicit TCPConnection(const TCPConfig &cfg) : _cfg{cfg} {}
91
92     //! \name construction and destruction
93     //! moving is allowed; copying is disallowed; default construction
       not possible
94
95     //!@{
96     ~TCPConnection();    //!< destructor sends a RST if the connection is
       still open
97     TCPConnection() = delete;
98     TCPConnection(TCPConnection &&other) = default;
99     TCPConnection &operator=(TCPConnection &&other) = default;
100    TCPConnection(const TCPConnection &other) = delete;
101    TCPConnection &operator=(const TCPConnection &other) = delete;
102    //!@}
103 };
104
105 #endif // SPONGE_LIBSPONGE_TCP_FACTORED_HH

```

2. libsponge/tcp_connection.cc

```

libsponge/tcp_connection.cc  cpp
1  #include "tcp_connection.hh"
2
3  #include <iostream>
4  #include <limits>
5
6  using namespace std;
7
8  size_t TCPConnection::remaining_outbound_capacity() const {
9      return _sender.stream_in().remaining_capacity();
10 }
11
12 size_t TCPConnection::bytes_in_flight() const {
13     return _sender.bytes_in_flight();
14 }
15
16 size_t TCPConnection::unassembled_bytes() const {
17     return _receiver.unassembled_bytes();
18 }
19
20 size_t TCPConnection::time_since_last_segment_received() const {
21     return _time_since_last_segment_received;

```

```

22  }
23
24  void TCPConnection::segment_received(const TCPSegment &seg) {
25      if (!_is_active) return;
26
27      _time_since_last_segment_received = 0;
28
29      // 1. 如果设置了RST标志，将入站流和出站流都设置为错误状态，并永久终止连接。
30      if (seg.header().rst) {
31          _unclean_shutdown(false);
32          return;
33      }
34
35      // 2. 把这个段交给TCPReceiver，这样它就可以在传入的段上检查它关心的字段：
36      // seqno、SYN、负载以及FIN。
37      _receiver.segment_received(seg);
38
39      // 3. 如果设置了ACK标志，则告诉TCPSender它关心的传入段的字段：ackno和
40      // window_size。
41      if (seg.header().ack) {
42          _sender.ack_received(seg.header().ackno, seg.header().win);
43      }
44
45      // 被动关闭检查：如果在出站流发送结束前，入站流已经全部接收完毕，则需要将
46      // _linger_after_streams_finish设置为false
47      if (_receiver.stream_out().input_ended() && !
48          _sender.stream_in().eof()) {
49          _linger_after_streams_finish = false;
50      }
51
52      // 4. 如果传入的段包含一个有效的序列号，TCPConnection确保至少有一个段作为应答
53      // 被发送，以反应ackno和window_size的更新。
54      // 5. 如果传入的段包含一个无效的序列号，这种段被称为“keep-
55      // alive”... TCPConnection应该回复这些“keep-alive”。
56      bool send_ack_needed = seg.length_in_sequence_space() > 0;
57
58      // 判断 keep-alive (seqno = ackno - 1, length = 0)
59      if (_receiver.ackno().has_value() &&
60          seg.length_in_sequence_space() == 0 &&
61          seg.header().seqno == _receiver.ackno().value() - 1) {
62          send_ack_needed = true;
63      }
64  }
65

```

```

59     if (send_ack_needed) {
60         _sender.fill_window();
61         if (_sender.segments_out().empty()) {
62             _sender.send_empty_segment();
63         }
64     }
65
66     _push_segments_out();
67 }
68
69 bool TCPConnection::active() const { return _is_active; }
70
71 size_t TCPConnection::write(const string &data) {
72     if (!_is_active) return 0;
73
74     size_t written = _sender.stream_in().write(data);
75     _sender.fill_window();
76     _push_segments_out();
77
78     return written;
79 }
80
81 ///! \param[in] ms_since_last_tick number of milliseconds since the last
call to this method
82 void TCPConnection::tick(const size_t ms_since_last_tick) {
83     if (!_is_active) return;
84
85     _time_since_last_segment_received += ms_since_last_tick;
86
87     // 1. 告诉TCPSender时间的流逝。
88     _sender.tick(ms_since_last_tick);
89
90     // 2. 如果连续重传的次数超过上限TCPConfig::MAX_RETX_ATTEMPTS，则终止连接，
并发送一个重置段给对端。
91     if (_sender.consecutive_retransmissions() >
        TCPConfig::MAX_RETX_ATTEMPTS) {
92         _unclean_shutdown(true);
93         return;
94     }
95
96     _push_segments_out();
97
98     // 3. 如有必要，结束连接。

```



```

99      // 需要满足四个条件：入站流已经全部接收完毕；出站流已经全部发送完毕；需要发送的数据对方已完全确认。
100     if (_receiver.stream_out().input_ended() &&
101         _sender.stream_in().eof() &&
102         _sender.bytes_in_flight() == 0) {
103
104         // _linger_after_streams_finish 为 false 时对应 d.ii, 立即结束连接。
105         if (!_linger_after_streams_finish) {
106             _is_active = false;
107         }
108
109         // _linger_after_streams_finish 为 true 时对应 d.i, 需要停留 10 *
110         // _cfg.rt_timeout 时间后结束
111         else if (_time_since_last_segment_received ≥ 10 *
112                 _cfg.rt_timeout) {
113             _is_active = false;
114         }
115     }
116 }
117
118 void TCPConnection::end_input_stream() {
119     _sender.stream_in().end_input();
120     _sender.fill_window();
121     _push_segments_out();
122 }
123
124 void TCPConnection::connect() {
125     _sender.fill_window();
126     _is_active = true;
127     _push_segments_out();
128 }
129
130 TCPConnection::~TCPConnection() {
131     try {
132         if (active()) {
133             cerr << "Warning: Unclean shutdown of TCPConnection\n";
134             _unclean_shutdown(true);
135         }
136     } catch (const exception &e) {
137         std::cerr << "Exception destructing TCP FSM: " << e.what() <<
138         std::endl;
139     }
140 }
141
142 }
143
144 }
145
146 }
147

```

```

138 void TCPConnection::_push_segments_out() {
139     while (!_sender.segments_out().empty()) {
140         TCPSegment seg = _sender.segments_out().front();
141         _sender.segments_out().pop();
142         size_t win_size = _receiver.window_size();
143         if (win_size > std::numeric_limits<uint16_t>::max()) {
144             seg.header().win = std::numeric_limits<uint16_t>::max();
145         } else {
146             seg.header().win = static_cast<uint16_t>(win_size);
147         }
148         if (_receiver.ackno().has_value()) {
149             seg.header().ack = true;
150             seg.header().ackno = _receiver.ackno().value();
151         }
152
153         _segments_out.push(seg);
154     }
155 }
156
157 void TCPConnection::_unclean_shutdown(bool send_rst) {
158     _sender.stream_in().set_error();
159     _receiver.stream_out().set_error();
160     _is_active = false;
161
162     if (send_rst) {
163         _sender.send_empty_segment();
164         if (!_sender.segments_out().empty()) {
165             TCPSegment rst_seg = _sender.segments_out().front();
166             _sender.segments_out().pop();
167             rst_seg.header().ack = false;
168             rst_seg.header().rst = true;
169             _segments_out.push(rst_seg);
170         }
171     }
172 }

```

3. apps/webget.cc

apps/webget.cc

cpp

```

1  #include "socket.hh"
2  #include "util.hh"
3  #include "tcp_sponge_socket.hh"
4

```

```

5  #include <cstdlib>
6  #include <iostream>
7
8  using namespace std;
9
10 void get_URL(const string &host, const string &path) {
11     // Your code here.
12
13     // You will need to connect to the "http" service on
14     // the computer whose name is in the "host" string,
15     // then request the URL path given in the "path" string.
16
17     // Then you'll need to print out everything the server sends back,
18     // (not just one call to read() -- everything) until you reach
19     // the "eof" (end of file).
20
21     cerr << "Function called: get_URL(" << host << ", " << path << ").\n";
22     FullStackSocket sock;
23     sock.connect(Address{host, "http"});
24
25     string request{"GET " + path + " HTTP/1.1\r\nHost: " + host +
26     "\r\nConnection: close\r\n\r\n"};
27     sock.write(request);
28
29     while (!sock.eof()) {
30         cout << sock.read();
31     }
32     sock.wait_until_closed();
33 }
34
35 int main(int argc, char *argv[]) {
36     try {
37         if (argc ≤ 0) {
38             abort(); // For sticklers: don't try to access argv[0] if
39             argc ≤ 0.
40         }
41
42         // The program takes two command-line arguments: the hostname and
43         // "path" part of the URL.
44         // Print the usage message unless there are these two arguments
45         // (plus the program name

```

```

43         // itself, so arg count = 3 in total).
44         if (argc != 3) {
45             cerr << "Usage: " << argv[0] << " HOST PATH\n";
46             cerr << "\tExample: " << argv[0] << " stanford.edu /class/
cs144\n";
47             return EXIT_FAILURE;
48         }
49
50         // Get the command-line arguments.
51         const string host = argv[1];
52         const string path = argv[2];
53
54         // Call the student-written function.
55         get_URL(host, path);
56     } catch (const exception &e) {
57         cerr << e.what() << "\n";
58         return EXIT_FAILURE;
59     }
60
61     return EXIT_SUCCESS;
62 }

```